PROGRAMAÇÃO I

ESTRUTURAS DE DADOS EM PYTHON

VOLUME II • CADERNO 05

PILHAS, FILAS, ENCADEAMENTO E HASH

Regras de entrada e saída, conexões entre elementos e localização por chave.

COMO ESTUDAR Comece pelo problema concreto, acompanhe o estado dos dados antes e depois de cada operação e só então leia o código. Execute os exemplos na ordem. Quando aparecer um termo técnico, ele será definido no próprio capítulo. Não é necessário decorar comandos antes de entender o comportamento.

CONTEXTO DIDÁTICO Os exemplos acompanham uma rede fictícia de lojas, com produtos, clientes, estoque, pedidos e atendimento. Os registros são exemplos de aula, não dados comerciais reais. Este caderno pressupõe os fundamentos de Python e o Caderno 01 (listas).

Série de cadernos independentes e complementares • Texto explicativo, exemplos executáveis e roteiros físicos de representação.

Mapa de aprendizagem deste caderno

Depois de dominar as coleções, o estudante passa a estudar estruturas cujo comportamento é definido por regras próprias: pilhas e filas; e estruturas clássicas construídas com nós e acesso por chave. Os exemplos enfatizam como a estrutura funciona, por que ela existe e qual é o limite das analogias físicas.

As representações em tabelas mostram o que o computador armazena; as demonstrações físicas mostram como uma pessoa poderia simular as mesmas operações com cartões ou colegas em posições fixas. **O número da posição não se move: o elemento é que muda.**

1. Estrutura de dados e tipo abstrato de dados

Uma lista é uma estrutura concreta de Python. Pilha e fila, por outro lado, são frequentemente ensinadas como **tipos abstratos de dados**: especificam quais operações devem existir e quais regras devem ser respeitadas, sem impor uma única maneira de implementação. Podemos implementar pilha com uma lista e fila com `deque`, mas as regras continuam sendo o aspecto principal.

Estrutura	Regra de saída	Analogias
Pilha	Sai o elemento inserido mais recentemente	Livros empilhados; histórico de desfazer
Fila	Sai o elemento inserido há mais tempo	Senha de atendimento; pedidos de impressão

Nem toda coleção ordenada é uma pilha ou fila. Se o programa permite retirar qualquer elemento a qualquer momento, está usando outras operações além da interface clássica. O objetivo do tipo abstrato é justamente restringir o acesso para que as regras tenham significado.

2. Pilhas: o último a entrar é o primeiro a sair

Na pilha, inserimos e retiramos elementos pelo mesmo lado, chamado **topo**. A política é LIFO, sigla de *last in, first out* — último a entrar, primeiro a sair. Pense em uma pilha de caixas: se uma nova caixa foi colocada por cima, ela precisa ser retirada antes da caixa que estava abaixo.

Operação	Pilha da base para o topo	Topo
Inicial	[]	vazia
push A	[A]	A
push B	[A, B]	B
push C	[A, B, C]	C
pop	[A, B]	B
push D	[A, B, D]	D

NA MESA Empilhe quatro livros identificados. Cada vez que alguém acrescenta um livro, anote “push”. Para retirar, a regra permite pegar somente o livro do topo. Registre a ordem de saída e compare com a ordem em que foram colocados.

```
pilha = []
pilha.append("A")  # push
pilha.append("B")
pilha.append("C")
retirado = pilha.pop()  # pop: C
print(retirado)
print(pilha[-1])  # consulta ao topo: B
```

O método `append()` acrescenta ao final, e `pop()` sem índice retira o último elemento. Assim, o final da lista representa o topo. Tentar executar `pop()` em uma pilha vazia gera `IndexError`: antes de retirar, devemos conferir se existe algum elemento.

```
if pilha:
    print("Saiu:", pilha.pop())
else:
    print("Pilha vazia")
```

3. Pilhas em situações reais: desfazer operações

Um editor pode guardar versões anteriores para implementar a operação de desfazer. Quando acontece uma mudança, salvamos o estado anterior em uma pilha. Ao desfazer, recuperamos o último estado salvo. Em uma aplicação real, o histórico pode ser mais sofisticado; o exemplo concentra-se apenas na regra LIFO.

```
texto = "Pedido recebido"
historico = []

historico.append(texto)
texto = "Pedido em separação"

historico.append(texto)
texto = "Pedido enviado"

texto = historico.pop()  # desfaz a última mudança
print(texto)  # Pedido em separação
```

A pilha armazena os estados anteriores, não comandos de texto. Se a ação de desfazer também precisar ser desfeita, poderíamos criar outra pilha para refazer; isso apresenta uma segunda estrutura coordenada com a primeira.

4. Filas: o primeiro a entrar é o primeiro a sair

Em uma fila, novos elementos entram pelo **final**, enquanto o atendimento ocorre pela **frente**. A política FIFO significa *first in, first out*: primeiro a entrar, primeiro a sair. O comportamento é adequado para senhas de atendimento sem prioridade especial.

Operação	Fila da frente para o final
Chega A01	[A01]
Chega A02	[A01, A02]
Chega A03	[A01, A02, A03]
Atende A01	[A02, A03]
Chega A04	[A02, A03, A04]
Atende A02	[A03, A04]

DINÂMICA COM PESSOAS Cinco alunos recebem senhas de atendimento. O primeiro ocupa a marca “frente”; novos clientes formam-se atrás dos demais. A cada comando “atender”, a pessoa que está na frente deixa a fila e a seguinte passa a ser a primeira. Senhas não devem ser trocadas sem uma regra adicional.

5. Implementação eficiente de fila com deque

Uma lista Python até pode representar fila usando `append()` e `pop(0)`, mas retirar a posição zero exige deslocar muitos elementos. A estrutura `collections.deque` foi preparada para inserções e remoções eficientes nas duas pontas. Para uma fila clássica, usamos `append()` para entrar e `popleft()` para sair.

```
from collections import deque
fila = deque()
fila.append("A01")
fila.append("A02")
fila.append("A03")
print(fila.popleft())  # A01
fila.append("A04")
print(fila[0])       # A02: próximo
```

Operação da fila	Código	Regra
Enfileirar	fila.append(item)	Adiciona ao final
Desenfileirar	fila.popleft()	Remove da frente
Próximo	fila[0]	Consulta sem remover
Quantidade	len(fila)	Conta elementos

Quando a fila está vazia, `popleft()` gera `IndexError`. Consulte `if fila` ou use uma verificação equivalente. Uma fila com prioridades não é necessariamente FIFO puro: se atendimentos urgentes podem passar na frente, precisamos definir outra política e talvez outra estrutura.

6. Pilha, fila e lista: comportamentos distintos

Mesmo que pilha e fila guardem elementos em sequência, seus contratos de uso diferem. A lista permite consultar índices variados; a pilha expõe principalmente o topo; a fila expõe principalmente a frente e a entrada pelo final. Essa distinção precisa aparecer nos projetos, porque a regra de uso pode ser mais importante que o recipiente escolhido.

Situação	Escolha indicada	Por quê
Desfazer últimas operações	Pilha	Recupera o estado mais recente primeiro
Atender por ordem de chegada	Fila	Mantém prioridade temporal
Acessar produto pelo índice 7	Lista	Permite acesso posicional
Pesquisar produto por código único	Dicionário	Facilita acesso por chave

7. Por que existem listas encadeadas?

Em uma lista sequencial, pensamos em posições numeradas em uma única coleção. Em uma **lista encadeada**, cada elemento é um **nó** que guarda um valor e uma referência para o próximo nó. Não precisamos representar uma sequência como uma coleção de posições contíguas; a ligação entre nós descreve a ordem.

Nó	Valor	Próximo nó
Nó A	Recebido	Nó B
Nó B	Separação	Nó C
Nó C	Enviado	None

A variável `inicio` referencia o primeiro nó. Para chegar ao terceiro, precisamos acompanhar a ligação do primeiro para o segundo e do segundo para o terceiro. Diferentemente do acesso `lista[2]`, não existe um salto direto para o terceiro nó em uma implementação simplesmente encadeada.

REPRESENTAÇÃO FÍSICA Três alunos seguram etiquetas A, B e C. Cada um aponta para o próximo; o último aponta para uma placa “None”. Quando o professor pede o elemento final, um colega precisa seguir as indicações do primeiro ao segundo e do segundo ao terceiro. Os alunos não precisam estar fisicamente lado a lado: as setas representam as conexões.

8. Construindo uma lista simplesmente encadeada

Em Python, podemos representar cada nó com uma classe pequena, cujo papel é guardar um valor e a ligação para o nó seguinte. Ainda não é um estudo completo de orientação a objetos: a classe aparece aqui somente para dar forma a uma estrutura clássica.

```
class No:
    def __init__(self, valor, proximo=None):
        self.valor = valor
        self.proximo = proximo

terceiro = No("Enviado")
segundo = No("Separação", terceiro)
primeiro = No("Recebido", segundo)

atual = primeiro
while atual is not None:
    print(atual.valor)
    atual = atual.proximo
```

O laço começa em `primeiro` e avança pela referência `proximo`. Quando essa referência é `None`, chegamos ao fim. A mesma ideia funciona com uma quantidade variável de nós: percorrer exige seguir as conexões, e o custo para chegar ao final cresce com o número de nós.

Inserir no começo

```
novo = No("Criado", primeiro)
primeiro = novo
print(primeiro.valor)          # Criado
print(primeiro.proximo.valor)    # Recebido
```

A inserção no início envolve criar um nó que aponta para o antigo primeiro e atualizar a referência de início. Se não precisarmos percorrer a lista nem procurar uma posição específica, essa operação é constante. Inserir em uma posição arbitrária exige alcançar seu antecessor, o que pode custar O(n).

9. Acesso sequencial versus acesso por chave: tabelas hash

Imagine procurar o estoque do produto de código 105. Uma lista de registros pode exigir comparar vários códigos. Em um dicionário, escrevemos `estoque_por_codigo[105]` para solicitar o valor associado diretamente à chave. Por trás de dicionários e conjuntos, Python utiliza estruturas de **tabela hash**.

```
estoque_por_codigo = {
    101: 18,
    103: 9,
    105: 32
}
print(estoque_por_codigo[105])  # 32
estoque_por_codigo[105] -= 2
print(estoque_por_codigo[105])  # 30
```

A ideia básica é transformar uma chave em uma informação numérica chamada **hash**, utilizada internamente para localizar uma região de armazenamento. A implementação real de um dicionário é mais complexa que o desenho didático de caixas numeradas, mas essa analogia ajuda a entender por que o acesso por chave costuma ser rápido.

10. Função hash e colisões

Duas chaves diferentes podem produzir o mesmo destino em uma tabela de tamanho limitado. Esse encontro recebe o nome de **colisão**. Colisão não significa necessariamente erro: a estrutura deve prever um método para resolvê-la. Um exemplo didático é calcular o resto da divisão do código por 5.

```
def indice_didatico(codigo, tamanho=5):
    return codigo % tamanho

for codigo in [101, 106, 103]:
    print(codigo, indice_didatico(codigo))
# 101 -> 1; 106 -> 1; 103 -> 3
```

Código	Resto da divisão por 5	Destino
101	1	Caixa 1
106	1	Caixa 1: colisão
103	3	Caixa 3

Uma forma didática de lidar com colisões é guardar, em cada caixa, uma lista de pares de chave e valor e percorrer essa lista ao buscar a chave desejada. É apenas uma estratégia; implementações reais podem usar outras técnicas, como endereçamento aberto.

```
tabela = [[] for _ in range(5)]
def guardar(codigo, quantidade):
    pos = codigo % len(tabela)
    for i, (chave, _) in enumerate(tabela[pos]):
        if chave == codigo:
            tabela[pos][i] = (codigo, quantidade)
            return
    tabela[pos].append((codigo, quantidade))

guardar(101, 18)
guardar(106, 5)
print(tabela[1])  # [(101, 18), (106, 5)]
```

O exemplo mostra **encadeamento por listas** dentro das caixas e funciona para guardar os pares. Não é uma reimplementação completa de `dict`: faltam busca segura, remoção, redimensionamento e outros cuidados de eficiência. O propósito é tornar visível o tratamento de uma colisão.

11. O que é hashable e por que isso importa?

Uma chave de dicionário precisa poder ser usada com hashing e comparação de igualdade consistente. Textos, números e muitas tuplas são adequados. Listas não podem ser chaves porque são mutáveis. Essa exigência evita que o conteúdo usado para localizar uma entrada mude de maneira incompatível com a posição de armazenamento.

```
estoques = {("Centro", 101): 18}
print(estoques[("Centro", 101)])

# Esta tentativa falharia com TypeError:
# errado = {["Centro", 101]: 18}
```

O custo médio de consulta em dicionários é geralmente **O(1)**, mas isso não é garantia absoluta de uma única comparação em todo caso. Colisões, tamanho e política de redimensionamento influenciam o desempenho. No próximo caderno, a notação Big O permitirá interpretar essas afirmações com cuidado.

Para guardar deste caderno

- **Pilha (LIFO)**: push/pop no topo; apropriada para histórico e desfazer.
- **Fila (FIFO)**: entrada no final e saída pela frente; `collections.deque` é implementação adequada em Python.
- **Lista encadeada**: nós conectados por referências; percorrer exige seguir os links.
- **Tabela hash**: procura chaves por uma transformação interna; colisões são esperadas e devem ser tratadas.
- Uma estrutura abstrata é definida por operações e regras, não por um único formato de código.

Referências de consulta

Python Software Foundation. The Python Tutorial: Data Structures. https://docs.python.org/3/tutorial/datastructures.html

Python Software Foundation. Biblioteca padrão: collections e json. https://docs.python.org/3/library/collections.html ; https://docs.python.org/3/library/json.html

VisuAlgo. Visualizações e explicações de algoritmos. https://visualgo.net/en

Harvard University. CS50x 2026, Week 3 — Algorithms. https://cs50.harvard.edu/x/weeks/3/

Bhargava, Aditya. Entendendo Algoritmos (Grokking Algorithms), 2ª edição. Manning Publications. Referência de abordagem visual; exemplos e explicações deste caderno são originais.